# Odoo ERP Client Provisioning and Production CI/CD Automation

Internship Presentation — B.Tech CSE (Cloud Computing)


**Student:** Kumar Nirupam
**Enrollment:** CS-23411229
**Batch:** 2026–27 | **Section:** 4CSE8

**Organization:** SystemaOps
**Role:** Cloud & DevOps Engineering Intern
**Institution:** IILM University, School of CSE

OVERVIEW

# Internship at a Glance

### Role

Cloud & DevOps Engineering Intern at SystemaOps — working on real production infrastructure

### Focus Areas

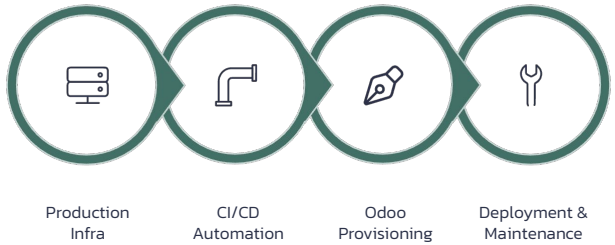
Containerized deployments, CI/CD automation, and Odoo ERP environment management

### Scale

Shared server infrastructure with live production deployments — not a sandbox environment

### Primary Project

Automated Odoo ERP Client Provisioning via GitHub Actions CI/CD pipeline



PROBLEM STATEMENT

# The Challenge: Manual Odoo Onboarding

Every new Odoo client required the same repetitive manual process — a bottleneck that introduced risk and inconsistency at production scale.

### Repeated Manual Configuration

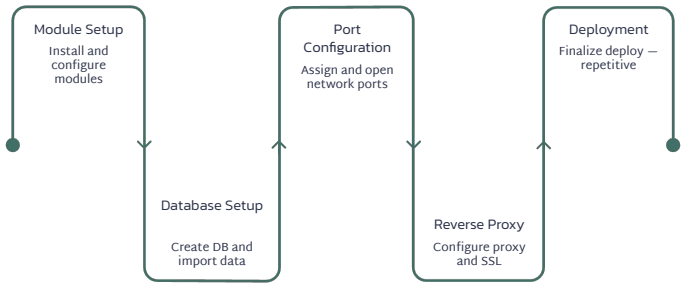
Each client onboarding demanded fresh manual setup of modules, database, ports, and routing from scratch

### Environment Complexity

Multiple production environments created conflicts around database isolation, port allocation, and Nginx routing

### Inconsistency Risk

Manual repetition increased operational effort and the likelihood of configuration drift across client environments



OBJECTIVES

# Project Objectives

### Automated Provisioning

Replace repetitive manual setup with a fully repeatable, workflow-driven provisioning system

### GitHub Actions CI/CD

Connect client requirements directly to automated workflow execution via GitHub Actions

### Centralized Modules

Use a Parent Odoo Application as the single source of truth for available client modules

### Client Isolation

Maintain separate, isolated Docker-based environments per client to prevent interference

### Database and Routing

Configure PostgreSQL and Nginx for per-client database access and reverse-proxy routing

### Production Reliability

Support deployment verification, automated backups, port management, and ongoing maintenance

TECHNOLOGY STACK

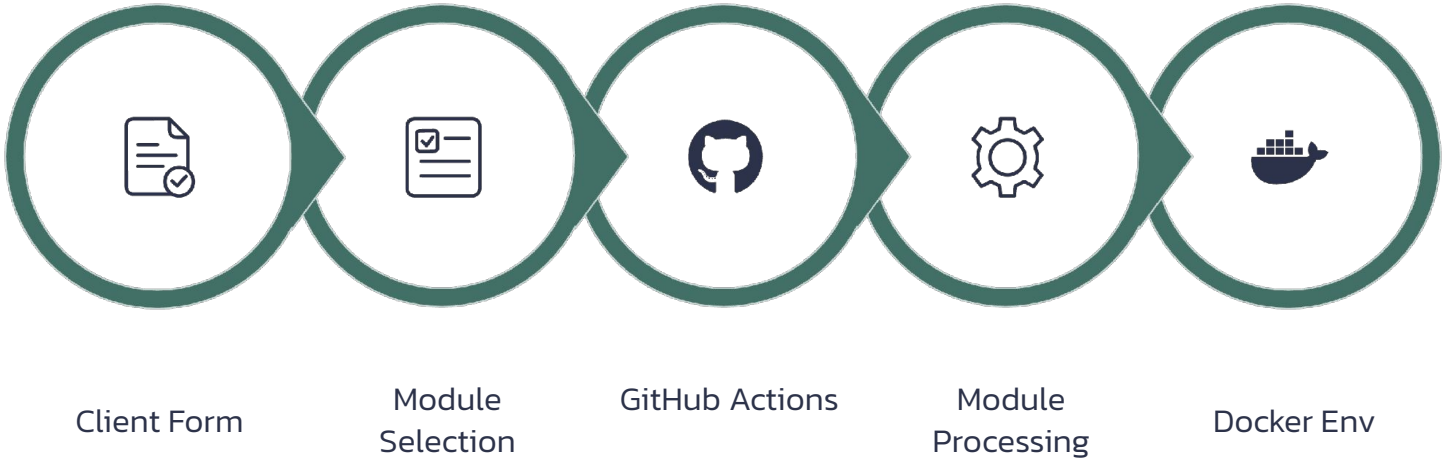
# Tools and Technologies

These tools supported the internship's production environment, selected for reliability, open-source flexibility, and compatibility with Linux-based infrastructure.

| | | | |
|---|---|---|---|
| **Infrastructure**<br>Linux / Debian | **Containerization**<br>Docker & Docker Compose | **CI/CD**<br>GitHub Actions | **ERP Platform**<br>Odoo 18 |
| **Database**<br>PostgreSQL | **Reverse Proxy**<br>Nginx | **Version Control**<br>Git & GitHub | **Automation**<br>Shell & Cron |

ARCHITECTURE

# High-Level Odoo Client Provisioning Architecture




*Customer requirements are transformed into a client-specific, containerized Odoo 18 environment through a fully automated CI/CD workflow.*

METHODOLOGY

# 8-Stage Provisioning Pipeline

01

### Requirement Analysis

Understand client needs and map to existing Odoo onboarding workflow

02

### Module Selection

Collect company information and identify required Odoo modules

03

### GitHub Actions CI/CD

Automate all provisioning steps through a GitHub Actions workflow

04

### Docker Provisioning

Create an isolated Odoo environment using Docker and Docker Compose

05

### Database & Reverse Proxy

Configure PostgreSQL and Nginx for the client-specific environment

06

### Verification & Troubleshooting

Check containers, DB connectivity, ports, routing, and application availability

07

### Maintenance & Backup

Maintain production services and automate Odoo backup schedules

08

### Continuous Improvement

Improve configuration reliability, resource usage, and operational efficiency

PRODUCTION DEPLOYMENT

# Production Odoo Infrastructure: 6 Client Instances

Six distinct industry verticals were deployed as fully isolated, production-grade Odoo 18 environments on shared server infrastructure.

Each Client Environment: Odoo 18 + PostgreSQL + Nginx

### Container Isolation

Each client runs in its own Docker container — no shared process space

### Port Management

Port assignments carefully tracked and allocated to prevent conflicts across instances

### Database Separation

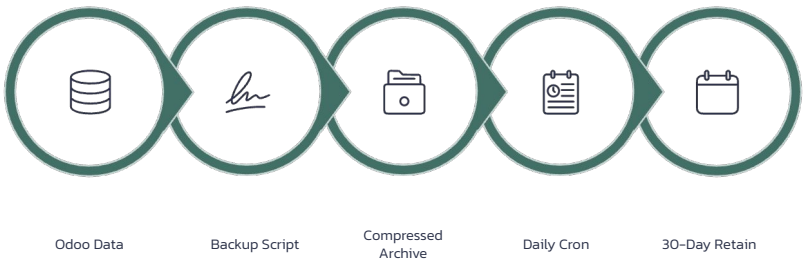
Independent PostgreSQL configurations maintained per client for full data isolation

### Nginx Routing

Reverse-proxy rules configured for client-specific URL access and traffic routing

RELIABILITY

# Production Reliability and Backup




## Backup System Highlights

### Automated Script

Odoo backup script created, tested, and deployed to production

### Daily Scheduling

Backup runs scheduled via Cron — no manual intervention required

### 30-Day Retention

Retention policy ensures old archives are cleaned automatically

### Audit Logging

Backup activity logged for verification and audit trail

### 3.1 GB Archive

Backup archive successfully created and validated in production

### Zero Disruption

Production environments maintained with no service interruptions

CI/CD AUTOMATION

# GitHub Actions CI/CD Automation


*Deployment reaches the live environment in approximately **3–8 minutes***

## How the CI/CD Pipeline Was Built

### 1 Automated Deployment Workflow

Built for the SystemaOps website — triggered automatically on code push

### 2 SSH Approach Blocked

Initial SSH-based deployment was blocked by external port 22 firewall restrictions

### 3 Self-Hosted Runner Solution

Switched to a GitHub Actions self-hosted runner installed directly on the production server

### 4 Systemd Service

Runner registered as a systemd service for auto-start and fault-tolerant operation

✓ 11 workflow runs successfully tested and deployed to the live production environment.

FINAL SLIDE

# Results

**50+**

Docker Containers

Managed across production

**6**

Odoo Instances

Live client environments deployed

**11**

Workflow Runs

CI/CD deployments executed

**~60GB**

Cache Recovered

Build cache cleaned and freed

**20+**

Issues Resolved

Production problems diagnosed and fixed

**3–8m**

Deploy Time

Code to live via CI/CD pipeline

# Thank You!